

4.1 Project Schedule: Gantt Chart

The project was planned and executed over a two-month window, from 01 July 2026 to 30 August 2026. The schedule below breaks the work into seven overlapping phases, sequenced so that design activities lead implementation and testing overlaps the tail end of development rather than following it strictly in series.

4.1.1 Phase Summary

Phase	Start Date	End Date	Duration (Days)
Requirements Gathering & Planning	01-Jul-2026	12-Jul-2026	12
System & Database Design	10-Jul-2026	24-Jul-2026	15
Frontend Development	20-Jul-2026	12-Aug-2026	24
Backend Development (API, Auth, Admin)	25-Jul-2026	12-Aug-2026	19
Payment Gateway Integration	01-Aug-2026	10-Aug-2026	10
Testing & Bug Fixing	08-Aug-2026	25-Aug-2026	18
Deployment & Documentation	20-Aug-2026	30-Aug-2026	11

4.1.2 Timeline

Phase	01 Jul	08 Jul	15 Jul	22 Jul	29 Jul	05 Aug	12 Aug	19 Aug	26 Aug
Requirements Gathering & Planning									
System & Database Design									
Frontend Development									
Backend Development (API, Auth, Admin)									
Payment Gateway Integration									
Testing & Bug Fixing									
Deployment & Documentation									

Note: Phase durations are planning-level estimates covering the 01 Jul 2026 – 30 Aug 2026 window, aligned with the project’s actual commit history — initial build-out from 31 Jul 2026, payment gateway integration around 05–06 Aug 2026, and iterative patches through 25 Aug 2026.

4.2 Data Design: Database and Schema Design

The Smart Event Management System uses MongoDB, a document-oriented NoSQL database, accessed through the Mongoose Object Data Modeling (ODM) library. Mongoose enforces schema-level structure, data types, and validation rules on top of MongoDB's otherwise schema-less documents, giving the application the flexibility of a document store while retaining the field-level guarantees typically associated with relational systems. The database is organized into four core collections — users, events, bookings, and payments — each backed by a dedicated Mongoose schema defined under `backend/models/`.

4.2.1 Database Overview

The table below summarizes the four collections that make up the application database, as observed in the deployed MongoDB Atlas cluster (`smart-event-db`):

Field	Type	Constraints	Description
users	Collection	Referenced by events, bookings, payments	Stores registered user and administrator accounts
events	Collection	References users (<code>createdBy</code>)	Stores event listings created by administrators
bookings	Collection	References users and events	Stores ticket bookings made against an event
payments	Collection	References users, events, and bookings	Stores payment transactions for a booking

Each schema is defined with the `{ timestamps: true }` option, so every document automatically carries `createdAt` and `updatedAt` fields that Mongoose maintains without application code.

4.2.2 Schema Definitions

4.2.2.1 User Schema

The `User` schema (`backend/models/User.js`) stores authentication credentials and the role that distinguishes a regular user from an administrator.

Field	Type	Constraints	Description
name	String	required, 2–50 chars	Display name of the account holder
email	String	required, unique, lowercase	Login identifier; validated against an email pattern
password	String	required, min 6 chars	Bcrypt-hashed password (never stored in plaintext)
role	String (enum)	'user' 'admin', default 'user'	Determines access to admin-only routes and pages
profileImage	String	optional, default ''	Stored image reference for the user avatar

Field	Type	Constraints	Description
createdAt / updatedAt	Date	auto-managed	Document creation and last-modified timestamps

A pre('save') hook hashes the password field with bcrypt (10 salt rounds) whenever it is created or modified, so plaintext passwords never reach the database. A comparePassword() instance method performs the corresponding bcrypt comparison during login, and a toSafeObject() method strips the password hash before any user document is sent to the client.

4.2.2.2 Event Schema

The Event schema (backend/models/Event.js) stores each event listing published on the platform.

Field	Type	Constraints	Description
title	String	required, max 120 chars	Event name
slug	String	required, unique	URL-friendly identifier derived from the title
description	String	required, max 1200 chars	Full event description
category	String	required, max 60 chars	Event category (e.g. Conference, Concert)
venue / city	String	required	Physical location of the event
organizerName	String	required, max 80 chars	Name of the organizing entity
imageData	String	optional	Base64-encoded event banner image
startDate / endDate	Date	required	Event start and end date/time
price	Number	required, min 0	Ticket price
totalTickets	Number	required, min 1	Total capacity at listing time
availableTickets	Number	required, min 0	Remaining unsold tickets; decremented on booking
status	String (enum)	draft published sold_out cancelled	Current lifecycle state of the event
featured / featuredUntil	Boolean / Date	optional	Controls homepage featured placement and its expiry
createdBy	ObjectId → User	optional	Reference to the administrator who created the event

4.2.2.3 Booking Schema

The Booking schema (backend/models/Booking.js) records a user's reservation of tickets for a specific event.

Field	Type	Constraints	Description
bookingReference	String	required, unique	Human-readable booking code, e.g. 'BKG-20260806-D5GIU2'
user	ObjectId → User	required	The user who made the booking
event	ObjectId → Event	required	The event being booked
eventTitle, eventSlug, eventStartDate, venue, city	String / Date	required, denormalized	Snapshot of event details at booking time
quantity	Number	required, min 1	Number of tickets booked
unitPrice / totalAmount	Number	required, min 0	Price per ticket and computed total
bookingStatus	String (enum)	pending_payment confirmed cancelled	Lifecycle state of the booking
paymentStatus	String (enum)	pending paid refunded not_required	Payment state associated with the booking
paymentProvider	String (enum)	demo upi_manual free manual	Payment channel used
paymentId / paymentOrderId	String	optional	External payment/order identifiers

Note that eventTitle, eventSlug, eventStartDate, venue, and city are intentionally duplicated from the Event document rather than resolved via a reference lookup at read time. This denormalization is discussed further in section 4.2.4.

4.2.2.4 Payment Schema

The Payment schema (backend/models/Payment.js) records the outcome of a payment attempt against a booking.

Field	Type	Constraints	Description
paymentReference	String	required, unique	Human-readable payment code, e.g. 'PAY-20260825-YZ0FKL'
user / event / booking	ObjectId	required	References tying the payment to its user, event, and booking
bookingReference / eventTitle	String	required	Denormalized snapshot fields for display without a join
provider	String (enum)	demo upi_manual free manual	Payment provider used for the transaction
method	String (enum)	demo_gateway upi_transfer offline none	Payment method used

Field	Type	Constraints	Description
amount	Number	required, min 0	Transaction amount
currency	String	default INR, uppercase	ISO currency code
status	String (enum)	pending paid refunded failed not_required	Current state of the transaction
transactionId	String	optional	Provider-side transaction reference
confirmedBy / confirmedAt	ObjectId → User / Date	optional	Admin who confirmed the payment, and when
refundedAt	Date	optional	Timestamp of refund, if applicable

The Payment schema additionally defines compound indexes on { booking, createdAt }, { user, createdAt }, and { status }, to keep the most common admin and user queries (a user's payment history, a booking's payment record, filtering by status) efficient as the collection grows.

4.2.3 Entity Relationships

The four collections form a linear reference chain rather than a fully normalized relational schema. Each downstream document stores an ObjectId reference back to its parent, while also carrying a denormalized snapshot of the parent's key display fields:

User (1) -----< Event.createdByUser (1) -----< Booking.user >-----
 Event (1) (Booking.event)Booking (1) ----- Payment.booking (Payment also stores user + event directly)

- A User can create many Events (via createdBy) and place many Bookings.
- An Event can be referenced by many Bookings (one per ticket purchase).
- A Booking has exactly one associated Payment document (a 1:1 relationship in current usage, modeled as a reference rather than embedding).
- A Payment redundantly stores user and event references directly, rather than requiring the application to traverse through Booking to know who paid or for what — trading a small amount of duplication for simpler, single-collection queries.

4.2.4 Design Rationale

Denormalization over joins

MongoDB does not perform relational joins efficiently at scale, and Mongoose's populate() (its join-equivalent) issues a separate query per reference. To avoid this cost on frequently read collections, Booking and Payment documents duplicate a handful of Event fields (title, slug, start date, venue, city) at the moment the document is created. This also means historical bookings and payments continue to display correct information even if an event's details are later edited — the booking reflects what the event looked like at the time of purchase, not its current state.

Security of credential storage

Passwords are never persisted in plaintext. The User schema's pre-save hook hashes every password with bcrypt before the document is written, and the comparePassword() method performs verification via bcrypt's constant-time comparison rather than a direct string match. The same users collection stores both regular users and administrators, distinguished only by the role field, rather than maintaining a separate admin table — role-based access control is enforced at the middleware/route level rather than through schema separation.

Enumerated status fields

Lifecycle-sensitive fields — Event.status, Booking.bookingStatus / paymentStatus, and Payment.status — are all constrained with Mongoose enum validators. This pushes state-machine validation into the schema layer itself, so an invalid status value (e.g. a typo) is rejected at the database-write stage rather than silently accepted.

Referential integrity without foreign-key constraints

MongoDB does not enforce foreign-key constraints natively. Referential correctness in this schema design is instead maintained at the application layer: controllers only create a Booking or Payment after confirming the referenced User and Event documents exist, and ObjectId reference fields are marked required so a dangling or missing reference cannot be persisted through the normal application flow.